Цифровая платформа ФСП: архитектура и проверка

Сопроводительная документация к локальному прототипу Срез проекта: 8 октября 2026 года Редакция: 1.2, локальная проверка завершена

> Это описание локального демонстрационного прототипа и результатов выполненных проверок. Синтетические профили, достижения ФСП, метрики и тестовые аккаунты не описывают реальных кандидатов, интеграцию с ФСП или качество найма. Техническое задание допускает синтетические данные для проверки; отсутствие человеческой калибровки честно раскрыто и не представляется как выполненная проверка.

1. Назначение и границы

Платформа реализует обратный сценарий найма: работодатель задаёт требования и изучает подходящих кандидатов, затем отправляет адресное приглашение с условиями. Кандидат сам решает, принимать ли приглашение. До принятия работодатель не получает контакты кандидата; после принятия доступ можно отозвать. Категория строится вокруг специализации и подтверждённого тестом грейда. Достижения ФСП служат дополнительным контекстом и ограниченным сигналом ранжирования, но не заменяют оценку компетенций.

Техническое задание ФСП требует описать функциональную и компонентную архитектуру, тестирование и подбор, валидацию с результатами, предлагаемую интеграцию с реестром, API, версии библиотек и шаги локального запуска (раздел 3.4). Раздел 3.5 задаёт слоистую архитектуру, OpenAPI, контроль доступа и воспроизводимый запуск; контейнеризация рекомендована. Текущее решение проверяется локально. Создание и публикация репозитория отложены пользователем.

2. Компонентная архитектура

Система - модульный сервер и отдельное web-приложение с общей PostgreSQL. В сервере границы модулей отделяют идентификацию и согласия, профиль, происхождение ФСП-данных, тестирование, подбор, приглашения и документы. HTTP-контроллеры проверяют запросы и права; прикладная политика формирует оценку и выдачу; JDBC/Flyway управляют транзакциями и схемой. Письма отправляет worker того же приложения через транзакционный outbox.

Компонент	Реализация и ответственность
Web-клиент	React: кабинеты кандидата и работодателя, оценка, профиль, поиск, списки и приглашения. Типы API генерируются из OpenAPI.
Backend/API	Kotlin + Spring Boot: сессии и роли, профиль и согласия, оценка, поиск/подбор, приглашения, права контактов и PDF-профиль.
Данные	PostgreSQL 17: пользователи, компании, профили, версии задач/политик, попытки, решения, потребности, приглашения и журнал согласий. Flyway применяет миграции.

Компонент	Реализация и ответственность
Email worker	Отдельный процесс того же Spring Boot приложения читает outbox с блокировкой и ограниченными повторами; в демо письмо принимает локальный SMTP sink.
Локальный контур	Compose описывает web, api, db, worker и demo-only Mailpit. Для нативного запуска используются локальный PostgreSQL 17, Spring и Vite.

Сессия непрозрачна для клиента и хранится на сервере; пароль проверяется Argon2id. Для изменяющих запросов применяются CSRF/Origin проверки, объектная авторизация и ключи идемпотентности. Контактные значения читаются только при активном разрешении пары «компания – кандидат». Выдача поиска и публичный профиль не включают закрытые контакты и заметки работодателя.

3. Оценка кандидата

Банк и состав формы

Первая категория – backend-python-v1, уровни junior и middle. Банк v1 содержит 4 блока компетенций × 2 уровня × 4 семейства = 32 семейства. Это один ограниченный пример специализации для MVP, не заявление о покрытии всего каталога ФСП или всех IT-ролей. Каждое семейство имеет два варианта A/B, по два вопроса в варианте: всего 64 варианта и 128 вопросов с одним выбираемым ответом. Блоки: Python, HTTP API, SQL и надёжность. Например, семейства проверяют работу с коллекциями и контрактами, HTTP-статусы и tenant isolation, SQL NULL/транзакции, дедлайны и идемпотентность.

Одна попытка содержит 3 разных семейства из каждого блока: 12 семейств и 24 ответа. За правильный ответ начисляется 1 балл, неправильный и пустой ответ дают 0. Максимум – 24 в сумме и 6 на блок.

Решение	Суммарный порог	Минимум в каждом блоке
Подтвердить выбранный уровень	18/24	3/6
Уверенное прохождение этой попытки	22/24	4/6

Уверенное прохождение не повышает грейд автоматически. Попытка длится 30 минут по серверному UTC-дедлайну; перезагрузка восстанавливает тот же снимок. Повтор той же цели доступен через 24 часа после валидной оценки. Изменение уже подтверждённого грейда ограничено cooldown 30 дней. Первая успешная фиксация не запускает cooldown; неудача не понижает сохранённый уровень. Невалидная из-за технического сбоя попытка помечается invalidated.

Разнообразие и ограничения

Сервер выбирает по три допустимых семейства на блок с учётом меньшей экспозиции кандидата и глобальной экспозиции; в каждом семействе выбирает A/B по тому же принципу и фиксирует случайный seed, порядок вариантов и снимок формы. Повторное открытие использует прежний снимок. Если собрать полную форму нельзя, зачёт не стартует. Это снижает повторяемость выдач, но не предотвращает утечку или запоминание заданий.

Банк имеет статус `draft_content_review`. Семейства и варианты не доказывают эквивалентность сложности, достаточность пула, психометрическую валидность или пригодность для найма. При четырёх семействах на блок после выдачи трёх остаётся только одно неиспользованное семейство в конкретной форме. До запуска реальной сертификации нужны человеческая предметная проверка, защищённый live-банк и эмпирическая калибровка. В демо контур блокирует присвоение реальных грейдов вне demo mode; встроенный seed с ответами предназначен для демонстрации и контент-ревью.

4. Подбор и категоризация

Категория - специализация плюс грейд. Целевой грейд выбирает кандидат, подтверждённый тестом уровень хранится отдельно от заявленного. Поиск не смешивает эти статусы. Работодатель может задавать требования к специализации, уровню, режиму/городу/часам и навыкам. Условия труда и неизвестные поля объясняются, а не сворачиваются в непрозрачный процент.

При подборе под сохранённую потребность результаты разделяются на три группы:

1. Подтверждённые: есть подтверждённый грейд и обязательные условия/свидетельства совпали. Только в этой группе FSP-сигнал может разрешать ничью после релевантных навыков.
2. Нужно уточнение: подтверждённая категория совпала, но требуемый атрибут профиля или обязательное свидетельство неизвестны. Неизвестное не считается несовместимостью и не получает уверенное подтверждение.
3. Нужно подтверждение: у кандидата нет подтверждённого тестом грейда; работодатель видит явную метку, а заявленный уровень остаётся самоописанием. Эти группы имеют стабильный порядок `confirmed → needs_clarification → needs_confirmation`.

Профиль без подтверждённой почты, без действующих согласий на обработку и публикацию, с закрытой видимостью или с явной несовместимостью обязательных условий исключается. Например, другой офисный город, несовместимый режим либо явно недостаточный максимум часов - hard mismatch. Неизвестный режим, город, часы или обязательный навык переводят профиль в уточнение, если это допускает политика. При строгом фильтре остаётся только первая группа.

Внутри подтверждённых сортировка лексикографическая: число желательных навыков, подтверждённых тестом; число желательных навыков из самоописания; допустимый FSP-сигнал; стабильный ID. В неполных группах используется покрытие подтверждённых обязательных и желательных навыков, затем заявленные желательные навыки и стабильный ID. Дублирующиеся навыки не считаются повторно; обязательный навык не получает второй бонус как желательный.

FSP-сигнал ограничен правилами происхождения: действительный связанный призовой результат - **2**, подтверждённое участие - **1**, нет подтверждённых данных - **0**; берётся максимум, а не сумма. Командный результат требует подтверждённого состава и не доказывает личный вклад. Если linkage участника отсутствует, сигнал не используется; публичная проекция помечает `used_for_ranking=false`. Дубли, престиж соревнования, регистрации и возраст не влияют. В неполных группах FSP не задаёт порядок. Нет данных ФСП означает «не подтверждено через ФСП», а не «низкая квалификация».

5. Предполагаемая интеграция с реестром ФСП

Техническое задание сообщает, что доступ к API и описание схемы реестра не предоставлены. Поэтому в прототипе используется явно помеченный mock/synthetic источник; рабочая интеграция и контракт ФСП API отсутствуют. Нельзя трактовать демонстрационные ID или достижения как факт реестра.

Предлагаемый адаптер отделяет доменную модель от будущего источника. После получения официального контракта оператор связывает профиль с FSP ID через согласованный механизм идентификации. Адаптер получает только разрешённые поля, сохраняет источник, внешний идентификатор, дату получения, версию схемы, статус подтверждения и признак командного состава. Сопоставление участника требует явной linkage-проверки; дублирующие и отозванные результаты не становятся сигналом. Отсутствующая история - отдельное нейтральное состояние. Сырые внешние данные не превращаются в грейд, а FSP ID и детали участия показываются по отдельным правилам видимости и согласия.

До реализации нужны: официальный API/выгрузка, правила идентификации и исправления ошибок, набор разрешённых полей, частота синхронизации, срок хранения и порядок отзыва. Keycloak/FSP ID совместимость можно добавить за адаптером после согласования. Здесь не заявляется существующий endpoint, структура API, доступ или согласованный регламент.

6. API и версии

HTTP-контракт описан в OpenAPI (`/api/v1`); успешные JSON-ответы используют `data`, ошибки - структурированный `error`, даты передаются в UTC, идентификаторы - UUID, страницы - через `limit/cursor`. API покрывает регистрацию/сессии, профиль и согласия, попытки оценки, потребности работодателя, поиск, списки, приглашения, контактные разрешения и PDF-проекции. Контакты не возвращаются до действующего grant; состояние попытки и оценка определяются сервером. Кандидатский endpoint не выдаёт ключи ответов и объяснения банка.

Срез **1.2** добавляет `POST /api/v1/candidates/search` для банка кандидатов без привязки к потребности: специализация, подтверждённый или заявленный уровень с явной меткой, статус категории, навыки и статус доказательств ФСП; ответ ограничен подтверждённой почтой, текущими согласиями и видимостью, не содержит контактов/заметок работодателя и помечен `private/no-store`. `GET/POST /api/v1/me/consents` возвращают собственные версии условий и принимают/отзывают отдельные scope обработки и публикации с проверкой роли и версии. Отзыв публикации скрывает профиль; отзыв обработки также блокирует дальнейшее редактирование и новые оценки до повторного согласия. Исторический флаг `visibility=public` не заменяет эти согласия: read gate требует текущие processing и publication acceptance, при этом запись профиля не меняется миграцией. Дополнительные добровольные поля профиля остаются заявленными и проходят очистку публичного текста. Поиск сохраняет read-only характер: курсор подписан HMAC и связан с компанией и фильтрами. Приглашение фиксирует snapshot контактных полей компании (отрасль, email, телефон), поэтому последующее редактирование профиля компании не меняет уже отправленные условия. Эти операции и DTO присутствуют в OpenAPI и коде текущей рабочей копии. Контракт содержит **46** путей, **55** операций и **82** схемы. Полная локальная проверка описана в разделе **7**.

Слой	Версия/инструмент
Backend	Kotlin 2.2.20, Spring Boot 3.5.9, Spring Dependency Management 1.1.7, Java 21, Gradle 8.14.5
Данные и безопасность	PostgreSQL 17.x, Spring JDBC/Flyway, Spring Security Crypto, BouncyCastle 1.81, JSON Schema Validator 1.5.6
Web	React/React DOM 19.2.0, TypeScript 5.9.3, Vite 8.0.0, React Router 7.9.4, TanStack Query 5.90.5, openapi-typescript 7.10.1
Web runtime	Node.js 24.x для локального окружения (проект требует Node не ниже 22.12)
Тесты	JUnit 5 через Spring Boot Test, PostgreSQL 17 для интеграционных тестов, Playwright
Документ PDF-профиля	Apache PDFBox 3.0.6 с локальным кириллическим шрифтом

Точные frontend-пакеты перечислены в frontend/package.json и frontend/package-lock.json; плагины/зависимости JVM задаются в backend/build.gradle.kts, wrapper закреплён на Gradle 8.14.5. Полный перечень разрешённых JVM runtime JAR с версиями вынесен в submission/dependency-versions.txt. Версии runtime-образов описаны в Dockerfiles: node:24-alpine, eclipse-temurin:21-jre, nginx:1.28-alpine, postgres:17-alpine и axllent/mailpit:v1.29. На другой машине проверьте эти файлы и используйте JDK 21.

7. Валидация и наблюдения

Синтетическая оценка запускалась через Spring HTTP/JDBC на PostgreSQL 17.10 fsp_test: 48 профилей, 96 попыток, 96 черновиков, 2304 ответа. Новое решение совпало в 43/48 пар (89,58%); расхождения сосредоточены у профилей около порога. Модель охватывает 64 элемента; strong/weak разница правильных ответов 0,375–1,000, среднее 0,621. Вероятности 0,35/0,75/0,95 заданы как предположения. Семейства использованы 36 раз каждое, варианты – 18 раз; между повторными формами совпали 8 семейств, одинаковых variant ID нет. Это синтетическая оценка вариативности, не калибровка грейда на людях.

Бенчмарк подбора matching-benchmark-v1: 12 потребностей × 20 профилей = 240 оценок, development/holdout по 6 потребностей и 120 оценок без пересечения ID и шаблонов. Политика получила P@3 0,8056 и NDCG@5 1,0000, сортировка только по заявленным навыкам – 0,5556 и 0,5191. Было 0 hard errors. AI-review 240 меток: PASS_PREMETRIC_AI_REVIEW, противоречий полей не найдено; это не human review и не проверка качества найма. SHA-256 frozen dataset: 127862d1cef31492cc2702904e13f7adbfed7f3f6f35a38519c258f0698abb3f. Синтетические метрики не устанавливают рыночную точность, fairness или SLA.

Свежая проверка 1.2: 107 backend tests, 0 failures/errors/skips; bootJar passed; frontend TypeScript и Vite build passed (npm run build). 13/13 локальных HTTP/Chromium сценариев прошли (отчёт local/verification/case-closure-runtime-2026-10-08.json): регистрация/email, оба согласия, дополнительные поля и PDF, оценка из 24 ответов, поиск/список, приглашение с company snapshot, контакт grant/revoke, отзыв/повтор публикации, 390 px mobile, плюс RecruitingRecord: без согласия имя скрыто, своя privateNote сохраняется, другой работодатель не создаёт запись. Отдельный backend-тест проверяет единый порядок блокировок user -> profile. В demo оценке было 24/24 и 6/6 по блокам; это синтетический результат. Mock ФСП показывался с provenance; без подтверждённой linkage/состава он не влияет на ранжирование.

Результаты подтверждают только локальные технические пути, не реальный найм, legal compliance или production readiness.

Финальное локальное видео `submission/FSP_Demo.mp4` заменяет публичный стенд: H.264/MP4, 155.705 секунд, 1600×1000; seek проверен. Публичное размещение не выполнялось.

8. Локальная сборка и запуск

Требования

JDK 21.

Node.js не ниже 22.12 (для воспроизводимой локальной среды используйте Node.js 24).

PostgreSQL 17; локальная конфигурация использует `fsp_dev`, интеграционные тесты - отдельную базу `fsp_test`.

Локальный SMTP sink на `127.0.0.1:1025` нужен для демонстрации доставки писем. Реальный внешний адресат не используется.

Нативный запуск

1. Скопируйте `.env.example` в локальный env-файл, заполните локальное подключение к PostgreSQL и экспортируйте переменные в shell. Не переносите секреты в публичный источник.
2. Установите зависимости фронтенда: `npm --prefix frontend ci`.
3. Запустите PostgreSQL 17 и убедитесь, что база `fsp_dev` доступна. Для подготовленной текущей станции можно использовать `./scripts/postgres-local.sh start`.
4. Запустите backend: `./scripts/backend-local.sh`. Приложение применит необходимые миграции согласно локальной конфигурации.
5. В другом терминале запустите web: `npm --prefix frontend run dev -- --host 127.0.0.1`.
6. Откройте `http://localhost:5173`. Демо-аккаунты и данные синтетические; пароли указаны в README и предназначены только для локального demo.
7. Для проверки email запустите SMTP sink, затем отдельный worker: `./scripts/backend-local.sh --spring.main.web-application-type=none --fsp.outbox-enabled=true --fsp.seed-demo=false`. Демо-Mailpit в Compose доступен на `http://localhost:8025`.

Сборка и проверки

```
./backend/gradlew -p backend --no-daemon --max-workers=1 test bootJar
npm --prefix frontend ci
npm --prefix frontend run types:api
npm --prefix frontend run typecheck
npm --prefix frontend run build
docker compose config --quiet
```

Чтобы запустить весь стек на новой машине с Docker Engine и Compose: `docker compose up --build`. UI доступен на `http://localhost:8080`; demo SMTP UI - на `http://localhost:8025`. Конфигурация Compose проверялась, но Docker daemon в описанном локальном срезе был выключен, поэтому подтверждением запуска является нативный контур, а не контейнерный runtime. Не направляйте тестовый backend на базу с реальными данными.

9. Сверка с требованиями кейса и раскрытия

Требование/ожидание	Где отражено	Статус и граница
Функциональная и компонентная архитектура (3.4)	Разделы 1-2	Описан текущий локальный модульный прототип.
Устойчивая механика оценки (3.4)	Раздел 3	Версионированный банк и вариативная выдача; утечка не исключена.
Подбор и категоризация (3.4)	Раздел 4	Три группы, hard filters, объяснимые источники признаков.
Валидация и результаты (3.4)	Раздел 7	ТЗ допускает синтетические данные и требует раскрыть процедуру/результаты; human validation не была обязательна для этой сдачи и не заявляется. AI-review указан отдельно от экспертной оценки.
Предполагаемая ФСП-интеграция (3.4)	Раздел 5	Только предложение через адаптер; реального API/данных нет.
API, версии библиотек, локальный запуск (3.4)	Разделы 6 и 8	OpenAPI, локальный build/run; Docker runtime не подтверждён.
Демонстрация сквозного сценария (5.2)	Локальное видео submission/FSP_Demo.mp4	H.264/MP4, 155.705 секунд, 1600×1000; seek проверен. Публичный стенд не развёрнут.
Открытый код и ссылка на репозиторий (3.5, 5.2)	Текущее состояние сдачи	Репозиторий отложен пользователем; ссылка не приводится. Это единственный незакрытый пункт чек-листа. Исходники доступны локально.
Контейнеризация и воспроизводимость (3.5)	Раздел 8	Compose описан и валидирован; нативные build/run и SMTP worker проверены, запуск контейнеров не заявляется.
Согласие, доступ и безопасность (3.5)	Разделы 2, 6 и 7	Согласие проверяется в public и RecruitingRecord проекциях и при создании записи; privateNote остаётся доступной только своей компании. Текст согласия - проект для review оператором, соответствие закону не заявлено.

Текст согласия и правила обработки в прототипе - проект для оператора. Этот документ не заявляет соответствие 152-ФЗ и не заменяет юридическую экспертизу, утверждение целей/оснований обработки, retention, порядка удаления, доступа оператора и обработки обращений. До работы с реальными персональными данными оператор должен утвердить документы и процессы.

Источники: docs/sources/fsp-specification-2026.md (3.4, 3.5, 5.1-5.2), README.md и документы docs/22, 23, 24, 26, 28, 31, 35, 36 (полные имена включают номер и дату).